

MICROSOFT FABRIC

End-to-End Data Platform

Lakehouse · Warehouse · Pipelines · Notebooks

MICROSOFT FABRIC

City Operations Data Platform

Public REST APIs to an analytics-ready star schema, loaded daily and checked at every layer

ORCHESTRATION · DATA PIPELINE
pl_city_ops_daily

Ingest 311 + Weather
2 notebooks in parallel

Transform to Silver
notebook

Load dimensions
stored procedure

Load facts
2 stored procedures

Reconcile
fails run on mismatch

On failure: log + fail
alert hook

Daily schedule · 06:00
Parameterised · retry policy

1 · SOURCES
Public REST APIs

NYC Open Data

311 service requests.
Socrata API, JSON,
50,000 rows per page.
Records change after
creation.

3.01M
requests in 2026

Open-Meteo

Hourly weather for the
five boroughs. Archive
plus recent observations,
which are revised.

33,195
hourly readings

2 · INGEST
PySpark notebooks

nb_01_ingest_nyc311
nb_02_ingest_weather
Settings come from a
control table.

Built to be re-run

High-watermark
incremental loads with
a look-back window.
Paging, retry and back-
off on throttling. Row
counts reconciled
against the API before
the watermark moves.

nb_00_common

One shared framework
for logging,
watermarks, retries and
quality checks.

3 · STORE AND REFINE · ONELAKE (DELTA)
Lakehouse lh_city_ops

LANDING

Raw API responses,
untouched

Files/Landing/n...
Files/Landing/w...

- Compressed
JSON
- Foldered by load
date and run id
- Replayable

86 files

BRONZE

As received, append-only
history

bronze.nyc311_service...
bronze.weather_hourly

- Every column kept as
text
- Lineage: file, run, load
time
- Partitioned by ingest
date

3.09M rows

SILVER

Typed, validated, one row per
business key

silver.service_requests
silver.weather_hourly
silver.service_requests...

- 10 data-quality rules,
results logged
- Bad rows kept in
quarantine
- Idempotent MERGE
upserts

3.01M requests

4 · MODEL · T-SQL
Warehouse wh_city_ops

dim_agency dim_complaint dim_channel

fact_service_request

dim_date dim_borough

fact_weather_daily

Stored procedures load only what changed,
inside a transaction. Two facts share
conformed date and borough dimensions.

3.01M fact rows · 5 dimensions

5 · SERVE
Analytics

Power BI

A Direct Lake semantic
model reads the Gold
tables directly, with no
import refresh.

SQL endpoint

Analysts query Gold
with plain T-SQL from
SSMS, Excel or any BI
tool.

Reporting
views

gold.vw_daily_borough
_summary

CONTROL AND AUDIT

Every run leaves a record

meta.source_config · meta.watermarks · meta.run_log · etl.load_log

Rows in, rows out, duration and status for each step. Sources are added or
tuned as data, without code changes.

DATA QUALITY

Rules with logged results

meta.dq_results

Each rule logs how many rows it checked and how many
failed, on every run.

RECONCILIATION

Layers must agree

etl.reconciliation_log

API vs Bronze and Silver vs Gold counts are compared. A
mismatch stops the run.

LIFECYCLE

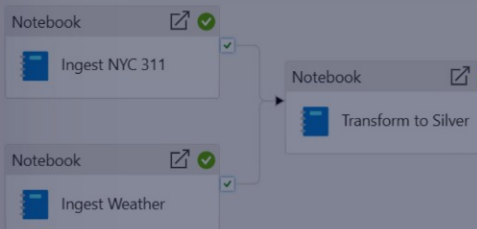
Everything is code

Notebooks, SQL, pipeline and semantic model are files,
deployed by script. Ready for Git and Dev / Test / Prod
promotion.

Monitoring hub > pl_city_ops_daily

Rerun Cancel Refresh Update pipeline List

Pipeline was modified after this run. The current pipeline configuration is sho



Activity runs

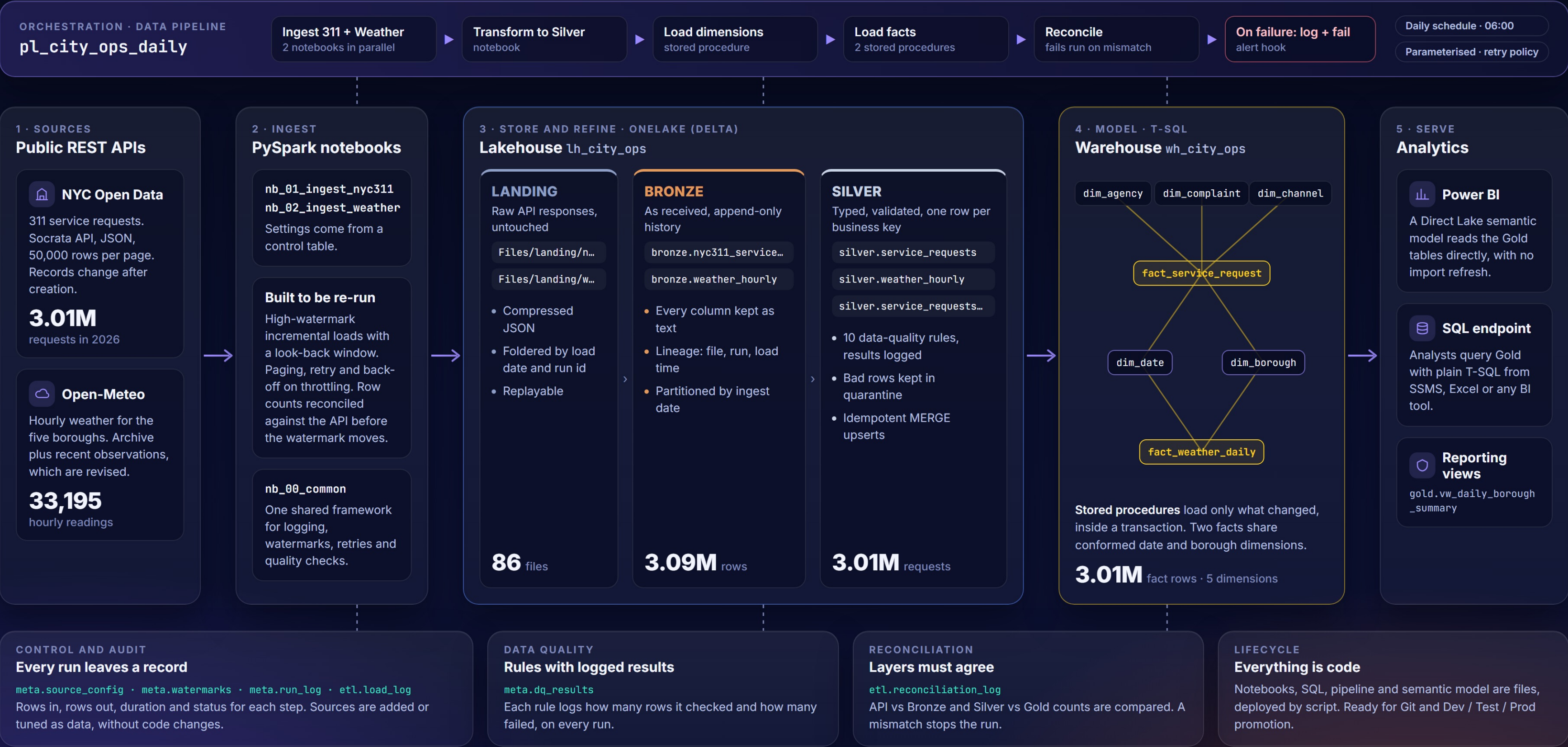
Filter by keyword Showing 1 - 7 items

Activity name	Activity status
Reconcile Silver vs Gold	Succeeded
Load Fact Weather Daily	Succeeded
Load Fact Service Request	Succeeded
Load Dimensions	Succeeded

City Operations Data Platform

Public REST APIs to an analytics-ready star schema, loaded daily and checked at every layer

— Data flow - - - - - Orchestration and control



One pipeline runs the whole load, every day

Ingestion, transformation, the warehouse load and the checks run as a single scheduled job, with one place to look when something goes wrong.

01 Each step waits for the one before

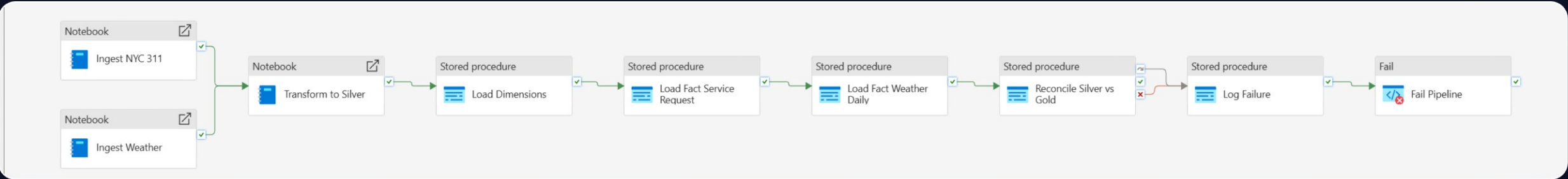
Each step starts only when the step before it has succeeded. The two API ingests run in parallel; the warehouse loads run in sequence to avoid write conflicts.

02 A failure is logged and fails the run

Any failed step triggers a branch that records the failure and marks the run as failed. An email or Teams alert plugs in at that point.

03 One parameter, two modes

`load_mode` switches the same pipeline between the daily incremental load and a full historical backfill.



Pipeline canvas. The last two steps are the failure branch.

Monitoring hub > pl_city_ops_daily

Rerun Cancel Refresh Update pipeline

ListGantt

Pipeline was modified after this run. The current pipeline configuration is shown.

Activity runs

Export to CSV Filter Column options

Showing 1 - 7 items

Activity name	Activity status	Run start	Duration	Input	Output
Reconcile Silver vs Gold	Succeeded	10/4/2026, 7:37:39 PM	9s		

Pipeline run details

Owner

Salman Hussain

Submitted by

Salman Hussain

Start time

10/4/2026, 7:31:19 PM

End time

10/4/2026, 7:37:50 PM

Status

Succeeded

Parameters

1

Pipeline run ID

bcaf0e24-1913-4f02-8ee7-88cf54d98e4f

6 min 46 s

full daily run, API to Gold

7 of 7

activities succeeded;
scheduled daily at 06:00

Monitoring hub. A real run: every activity with its status and duration.

APIs are read safely and landed untouched

The NYC 311 API returns 50,000 rows per page, and its records keep changing after they are created. The ingest is built around both facts.

01 Incremental by watermark

Only records created or updated since the last run are requested, with a short look-back for late changes. The daily run moved 36,400 rows instead of 3 million.

02 API calls that retry

The notebook pages through the API and splits a backfill into monthly windows that run in parallel. When the API throttles, times out or returns a server error, the call is retried with exponential back-off.

03 Row counts checked before the watermark moves

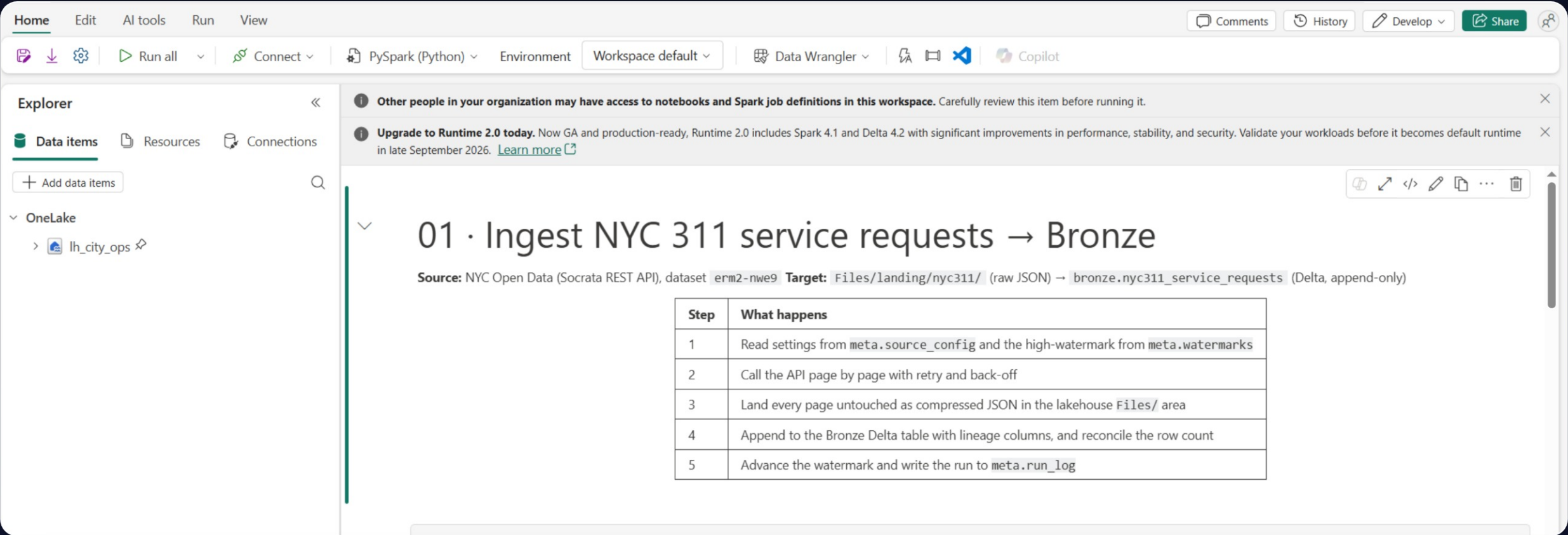
Rows returned by the API must equal rows landed in Bronze. Only then does the watermark advance, so a failed run simply retries from the same point.

3,014,623

rows backfilled in 21 minutes, reconciled to the row

42 s

daily incremental ingest



Notebook header. Each notebook documents its own source, target and steps.

```
7         Pull every page for one window and land each page as newline-delimited JSON (gzip).
8     name, where = window
9     offset, part, rows = 0, 0, 0
10    while True:
11        page = http_get_json(session, cfg["base_url"],
12                               {"$where": where, "$order": "unique_key", "$limit": page_size, "$offset": offset})
13        if not page:
14            break
15        part += 1
16        with gzip.open(f"{landing_local}/{name}_part-{part:04d}.json.gz", "wt", encoding="utf-8") as fh:
17            for record in page:
18                fh.write(json.dumps(record) + "\n")
19        rows += len(page)
20        offset += page_size
21        if len(page) < page_size:
22            break
23    print(f" window {name}: {rows:,} rows in {part} file(s)")
```

Extraction loop. Page, land as compressed JSON, repeat. Failures are logged and re-raised.

Three layers, each with one job

Raw data is never edited, and clean data is never mixed with raw. Every layer is an open Delta table that Spark, SQL and Power BI read without copies.

01 Landing and Bronze: the record of what arrived

Raw API responses are kept as files, then appended to Bronze with the file name, run id and load time on every row. Any number can be traced back to its source.

02 Silver: one trusted row per request

Typed columns, standardised values and a MERGE on the business key. When a request is updated at source, the existing row is updated, so there is still one row per request.

03 Control tables sit beside the data

The `meta` schema holds source settings, watermarks, the run log and data-quality results, all queryable with SQL.

3.09M

Bronze rows: full history of every load

3.01M

Silver rows: one per service request

HomeMaterialized lake views

Analyze data with

Share

Get data

New Spark SQL query

New semantic model

Add to data agent

Manage OneLake security

Update all variables

For optimal refresh to work efficiently, enable the delta CDF property on both source tables and materialized lake views used as source.

A SQL analytics endpoint for SQL querying was created with this item.

Explorer

Add lakehouses

lh_city_ops

Tables

dbo

bronze

nyc311_service_...

weather_hourly

meta

silver

service_requests

service_request...

weather_hourly

Files

landing

Files

nyc311_service_r...

weather_hourly

service_requests

Showing 1000 rows

Table view

	12L	unique_key	created_at	closed_at	due_at	resolution_u...	ABC agency_code	ABC agency_name	ABC complaint_t...	ABC descriptor	ABC lo
1		68113521	2026-02-24T07:...	2026-03-03T00:...	2026-03-26T07:...	2026-03-04T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
2		68713187	2026-04-18T06:...	2026-04-22T00:...	2026-05-18T06:...	2026-04-23T09:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
3		69910606	2026-07-31T10:...	2026-08-12T00:...	2026-08-30T10:...	2026-08-13T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
4		69664129	2026-07-09T09:...	2026-07-21T00:...	2026-08-08T09:...	2026-07-23T10:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
5		69143129	2026-05-27T09:...	2026-05-29T00:...	2026-06-26T09:...	2026-05-30T13:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
6		68728061	2026-04-20T08:...	2026-04-22T00:...	2026-05-20T08:...	2026-04-23T09:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
7		69676252	2026-07-10T20:...	2026-07-17T00:...	2026-08-09T20:...	2026-07-18T07:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
8		70237089	2026-08-29T22:...	2026-09-02T00:...	2026-09-28T22:...	2026-09-03T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
9		68577393	2026-04-06T08:...	2026-04-08T00:...	2026-05-06T08:...	2026-04-09T09:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
10		69147677	2026-05-27T09:...	2026-07-21T00:...	2026-06-26T09:...	2026-07-23T10:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
11		70152078	2026-08-22T11:...	2026-08-31T00:...	2026-09-21T11:...	2026-09-01T14:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
12		68242617	2026-03-07T07:...	2026-03-11T00:...	2026-04-06T07:...	2026-03-12T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
13		68796493	2026-04-26T08:...	2026-05-01T00:...	2026-05-26T08:...	2026-05-04T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL
14		68959696	2026-05-11T08:...	2026-05-13T00:...	2026-06-10T08:...	2026-05-14T06:...	DSNY	Department of S...	Graffiti	Graffiti	NULL

Succeeded (1 sec 380 ms)

Columns 30 Rows 1,000

Lakehouse explorer. Bronze, Silver and control schemas, with a preview of the typed Silver table.

Every bad row is counted and kept

Real public data is messy. The backfill contained requests closed before they were opened, and thousands with no borough. Each problem is handled by a named rule.

01 Rules run on every load

Ten rules check keys, dates, locations and duplicates each time data moves from Bronze to Silver.

02 Rejected rows go to quarantine

Rows that cannot be trusted go to a quarantine table with the reason. Rows with one bad field are repaired and flagged, so analysts can include or exclude them.

03 Rule results are stored

Every rule writes how many rows it checked and how many failed to `meta.dq_results`, so quality can be trended and alerted on like any other metric.

rule	check	when it fails	rows checked	rows failed	
SR01	Request id is present and numeric	Quarantine	3,014,623	0	clean
SR02	Created date is present, not in the future	Quarantine	3,014,623	0	clean
SR03	One row per request id	Keep latest	3,014,623	0	clean
SR04	Closed date is not before created date	Clear and flag	3,014,623	935	repaired
SR05	Closed date is not in the future	Clear and flag	3,014,623	1	repaired
SR06	ZIP code is five digits	Clear and flag	3,014,623	1	repaired
SR07	Coordinates fall inside New York City	Clear and flag	3,014,623	0	clean
SR08	Borough is one of the five boroughs	Set to Unknown, flag	3,014,623	3,286	repaired

Results of the backfill run, read from `meta.dq_results`. Two more rules cover the weather feed.

03 · Bronze → Silver: clean, validate, conform

Input:

`bronze.nyc311_service_requests`, `bronze.weather_hourly` (only rows not yet processed)

Output:

`silver.service_requests`, `silver.weather_hourly` (typed, de-duplicated, merged)

Rule	Check	When it fails
SR01	<code>unique_key</code> is present and numeric	Row goes to quarantine
SR02	<code>created_date</code> is present and not in the future	Row goes to quarantine
SR03	One row per <code>unique_key</code>	Latest version kept
SR04	<code>closed_date</code> is not before <code>created_date</code>	Closed date cleared, row flagged
SR05	<code>closed_date</code> is not in the future	Closed date cleared, row flagged
SR06	ZIP code is 5 digits	ZIP cleared, row flagged

Silver notebook. The rules are documented where they are implemented.

4,223

rows repaired and flagged in the backfill

0

rows lost between layers

A star schema the business can query

Gold is a dimensional model in the Fabric Warehouse: two fact tables sharing date and borough dimensions, loaded by stored procedures.

01 Built for reporting

Dimensions use surrogate keys, and each has an Unknown member so no fact row is dropped. The 197 raw complaint types roll up into nine reporting categories.

02 Incremental and transactional

Only changed rows are replaced, in one transaction with the watermark. The daily run updated 36,203 requests and added 197 new ones in 3 seconds.

03 Reconciled after every load

Five reconciliation checks compare Silver with Gold after every load. If they disagree, the run fails and reports are not refreshed on bad data.

3,014,820

Silver rows = Gold rows,
checked every run

5 of 5

reconciliation checks passed

HomeManagementHelp

Analyze data withShare

Get dataNew SQL querySQL templatesMonitorNew semantic modelNew API for GraphQLAdd to data agentDownload SQL database projectOpen inCopilot

SQL query 1

RunSave asSave as viewCopy queryCreate ruleExplain queryFix query errors

PreviewCopilot uses AI. Mistakes can happen. Verify code suggestions before running them. Review terms

```
1 -- Do heating complaints follow the temperature? One query on the Gold layer.
2 SELECT year_month,
3 CAST(AVG(avg_temperature_c) AS DECIMAL(4,1)) AS avg_temp_c,
4 SUM(total_requests) AS total_requests,
5 SUM(heat_requests) AS heat_complaints,
6 SUM(noise_requests) AS noise_complaints,
7 CAST(AVG(avg_resolution_hours) AS DECIMAL(6,1)) AS avg_resolution_hours
8 FROM gold.vw_daily_borough_summary
9 WHERE full_date >= '2026-01-01'
10 GROUP BY year_month
11 ORDER BY year_month;
```

MessagesResultsGridTableChart

	ABC year_month	e ^x avg_temp_c	123 total_requests	123 heat_complaints	123 noise_complaints	e ^x avg_resolution_hours
1	2026-01	-2.2	348511	79928	45540	297.7
2	2026-02	-2.9	334691	59031	42039	229.7
3	2026-03	6.6	342388	29783	59817	189.1
4	2026-04	12.5	302191	19553	60980	189.4
5	2026-05	17.5	331980	9823	78946	176.9
6	2026-06	23.4	334834	4388	76637	145.3
7	2026-07	25.1	343123	4331	70409	157.0
8	2026-08	24.3	328904	3682	70404	117.3
9	2026-09	20.3	323608	4862	69430	48.1
10	2026-10	20.7	24393	438	5333	3.2

AutoSave: On (Saved)Succeeded (1 sec 777 ms)Copilot completions: OffCode assistance: objects loadedColumns: 6 Rows: 10

One query on Gold answers a business question. Heating complaints: 79,928 in January at -2.2 °C, 3,682 in August at 24.3 °C. Noise complaints move the other way.

Ready for Power BI, traceable end to end

A semantic model sits on top of Gold for Power BI, and Fabric shows the full path from each report back to the source.

01 Direct Lake semantic model

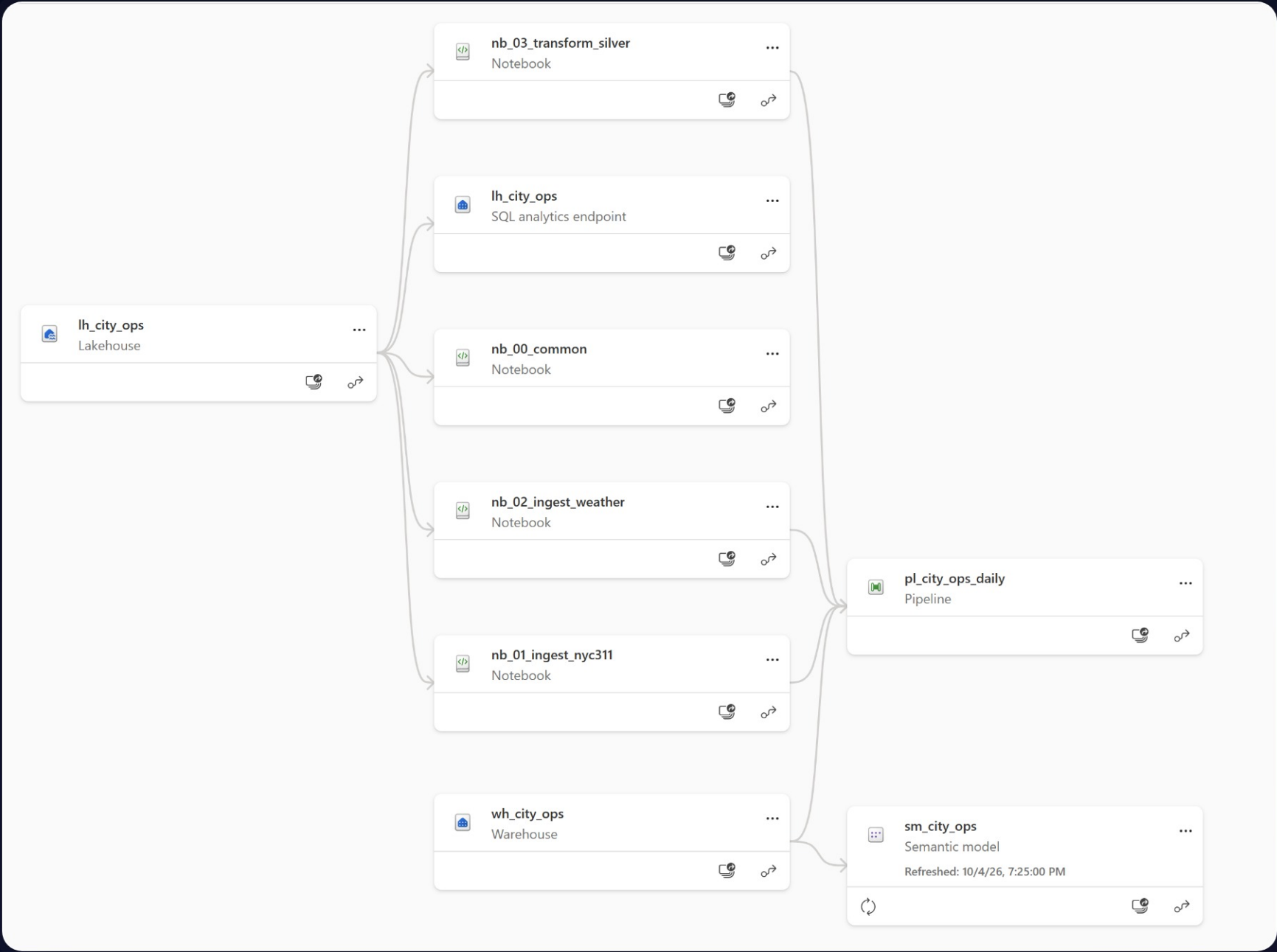
Power BI reads the Gold Delta tables directly, so there is no scheduled import and no second copy of the data. Reports show the latest load as soon as the pipeline finishes.

02 Measures defined once

Total requests, closure rate, average resolution time and weather measures live in the model, so every report uses the same definitions.

03 Lineage is visible in Fabric

Fabric draws the chain from lakehouse to notebooks, pipeline, warehouse and semantic model, which makes impact analysis quick.



Workspace lineage view. Lakehouse, notebooks, pipeline, warehouse and semantic model.

7

tables and 8 measures in the model

0

data copies between Gold and Power BI

What you get with this build

The same engineering standards apply whether the source is a public API, an ERP, a CRM or a folder of spreadsheets.

✓

Incremental loads
High-watermark with look-back. The daily run moved 36,400 changed rows out of 3 million.

✓

Safe to re-run
MERGE upserts and transactional loads. Running a load twice gives the same result.

✓

Data-quality rules
Ten named rules, results logged on every run, bad rows quarantined.

✓

Reconciliation gates
API vs Bronze and Silver vs Gold are compared. A mismatch fails the run.

✓

Full audit trail
Every step records rows in, rows out, duration and status.

✓

Failure handling
Retries on transient errors, a failure branch, and a hook for alerts.

✓

Settings held as data
Endpoints, page sizes and look-back windows live in a control table.

✓

Everything as code
Notebooks, SQL, pipeline and model are files, deployed by script.

2
REST API sources

3.01M
service requests loaded

6 min 46 s
daily end-to-end run

10
data-quality rules

100%
of rows reconciled across layers